

01 向量

1.7 投影

标量投影给坐标，向量投影给方向，正交补留下无法表达的剩余



鸢尾花书 · 《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

掌握标量投影的定义： $\mathbf{a}$ 在 $\mathbf{b}$ 方向上的标量投影 = $\|\mathbf{a}\|\cos\theta = \mathbf{a}\cdot\mathbf{b}/\|\mathbf{b}\|$ ，是一个标量“坐标值”

3

理解正交补的含义：向量中无法被投影方向线性表达的剩余部分，与投影方向垂直

5

理解标准正交基、基底、基向量、线性无关、线性组合等基础概念

2

掌握向量投影的定义：向量投影 = 标量投影 $\times$ 方向向量，同时给出“坐标 + 方向”

4

掌握向量正交分解：将一个向量拆分为投影与正交补两个相互垂直的分量之和

6

会用标量投影计算向量在任意“旋转正方网格”（旋转直角坐标系）下的坐标

本节知识脉络



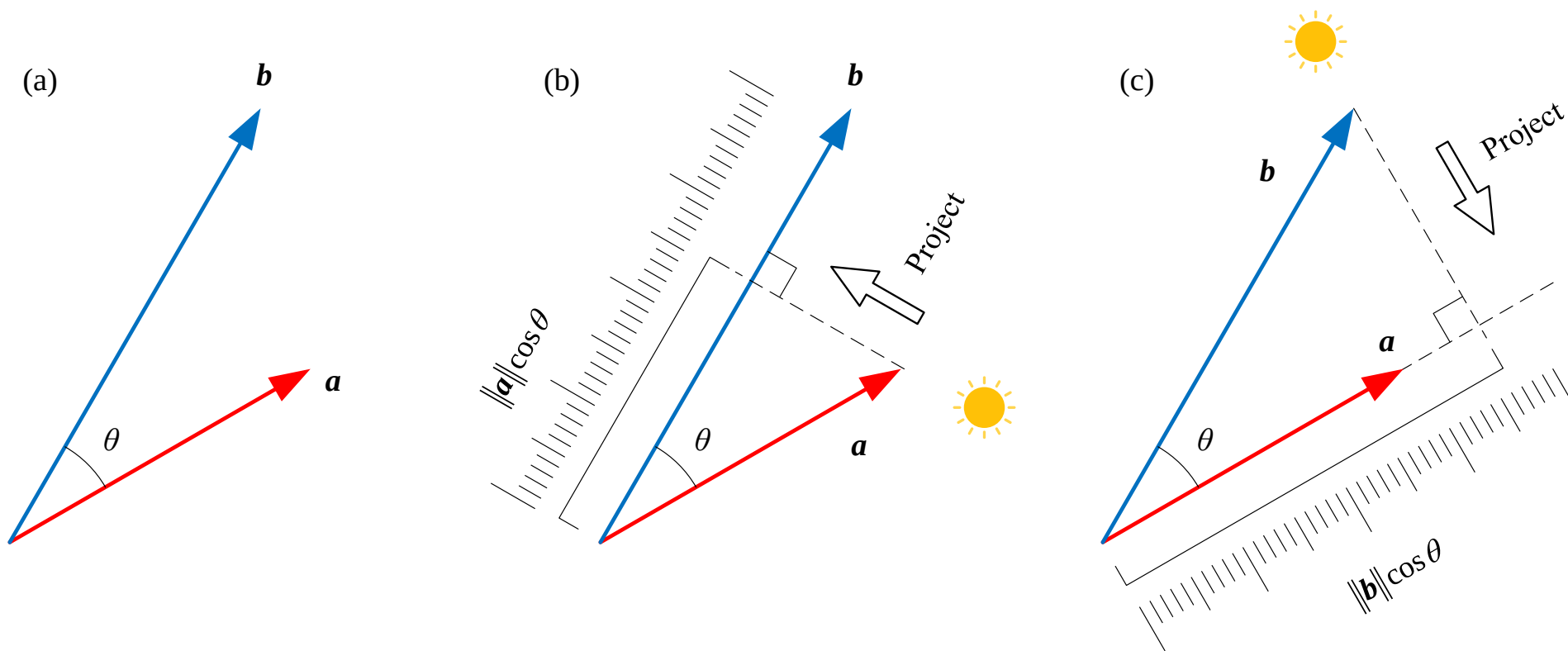
标量投影：向量在某方向上的“坐标”

$$\mathbf{a} \text{ 在 } \mathbf{b} \text{ 方向上的标量投影} = \|\mathbf{a}\| \cos \theta = \mathbf{a} \cdot \mathbf{b} / \|\mathbf{b}\|$$

- 给定非零向量 $\mathbf{a}$ 、 $\mathbf{b}$ ，两者夹角为 θ ；标量投影结果为一个标量，不包含方向信息
- 可以理解为向量 $\mathbf{a}$ 在向量 $\mathbf{b}$ 方向上正交投影的“坐标值”，即 $\mathbf{a}$ 在 $\mathbf{b}$ 方向上的“影子”长度
- 反过来， $\mathbf{b}$ 在 $\mathbf{a}$ 方向上的标量投影为 $\|\mathbf{b}\| \cos \theta = \mathbf{a} \cdot \mathbf{b} / \|\mathbf{a}\|$ ，两者互为对称

图1 • 标量投影的两个方向

- (a) θ 为向量 a 、 b 之间的夹角
- (b) $\|a\|\cos\theta$ 为 a 在 b 方向上的标量投影，虚线垂直于 b ，如同阳光下测量影子长度
- (c) $\|b\|\cos\theta$ 为 b 在 a 方向上的标量投影，虚线垂直于 a



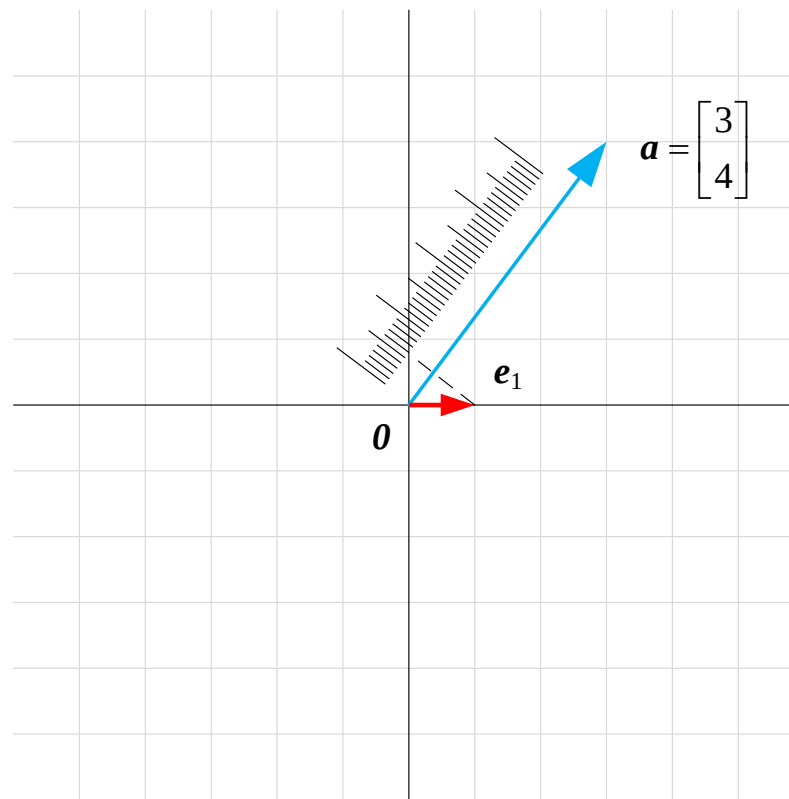
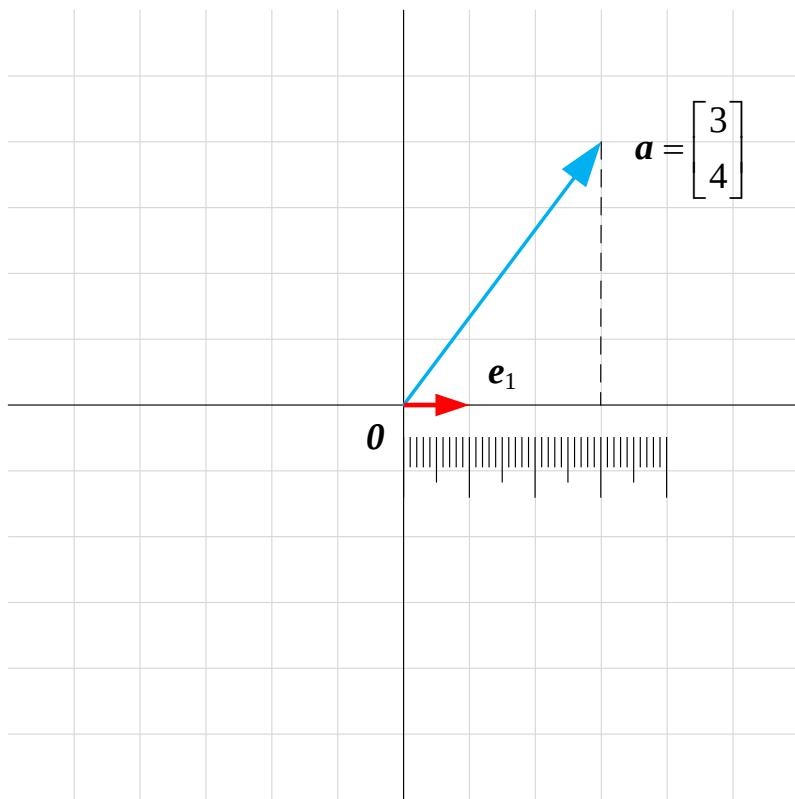
标量投影计算示例

$$\mathbf{a}=[3,4]^T \text{ 在 } \mathbf{e}_1 \text{ 方向上的标量投影} = \mathbf{a} \cdot \mathbf{e}_1 = 3$$

- $\mathbf{e}_1=[1,0]^T$ 本身是单位向量，因此可以直接用内积计算标量投影，无需再除以长度
- 反过来， $\mathbf{e}_1$ 在 $\mathbf{a}$ 方向上的标量投影为 $\mathbf{e}_1 \cdot (\mathbf{a} \text{ 的方向向量}) = 3/5$
- $\mathbf{a}$ 的长度为 5，因此 $\mathbf{e}_1$ 在 $\mathbf{a}$ 方向上的“影子”占 $\mathbf{a}$ 长度的 3/5

图2 · 向量 $\mathbf{a}$ 、 $\mathbf{e}_1$ 相互标量投影

- 左图： $\mathbf{a}=[3,4]^\top$ 在 $\mathbf{e}_1$ 方向上的标量投影为 3，即虚线垂直于 x_1 轴处的坐标值
- 右图： $\mathbf{e}_1$ 在 $\mathbf{a}$ 方向上的标量投影为 $3/5$ ，图中斜向刻度尺表示投影方向



NumPy 计算标量投影

内积除以长度，得到标量投影

- 用 `np.array()` 定义向量 `a=[3,4]`、`e1=[1,0]`
- `np.dot(a, e1)` 与 `np.dot(e1, a)` 结果相同，验证内积的交换律
- `np.linalg.norm()` 分别计算 `a`、`e1` 的长度
- 标量投影 = 内积 / 目标方向向量的长度，分别得到 `a` 在 `e1`、`e1` 在 `a` 方向上的投影

```
## 初始化
import numpy as np

## 定义向量
a = np.array([3, 4])
e1 = np.array([1, 0])

## 计算点积
dot_a_e1 = np.dot(a, e1)
dot_e1_a = np.dot(e1, a)

## 计算向量模长
norm_a = np.linalg.norm(a)
norm_e1 = np.linalg.norm(e1)

## 计算标量投影
scalar_proj_a_on_e1 = dot_a_e1 / norm_e1 # a 在 e1 上的投影
scalar_proj_e1_on_a = dot_e1_a / norm_a # e1 在 a 上的投影
```



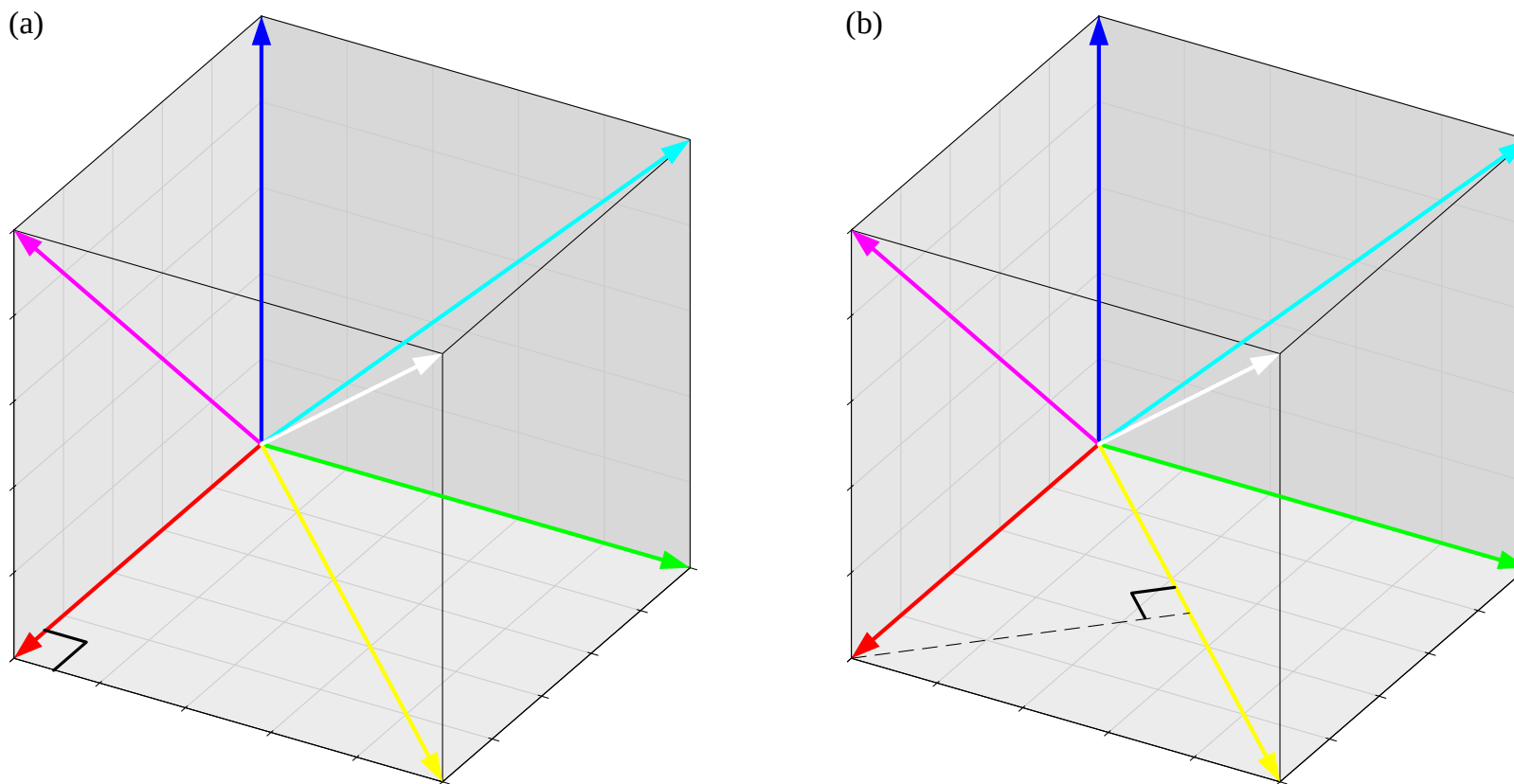
黄光在红光方向上的标量投影

$$[1,1,0]^T \cdot [1,0,0]^T / \|[1,0,0]^T\| = 1$$

- 黄光穿过红色滤光片后只保留红光强度，这个强度值即黄光向红色轴的标量投影
- 黄光向量 $[1,1,0]^T$ 在红光向量 $[1,0,0]^T$ 上的标量投影为 1，说明黄光中有 100% 强度的红光
- 反过来，红光在黄光上的标量投影为 $\sqrt{2}/2$ ；黄光长度为 $\sqrt{2}$ ，红光在其中占了 $1/2$

图3 • 黄光、红光的相互标量投影

- (a)(b) 两个视角展示 RGB 空间中红、绿、蓝、青、品红、黄、白等颜色向量的相互关系
- 请大家类比计算品红、蓝光的相互标量投影



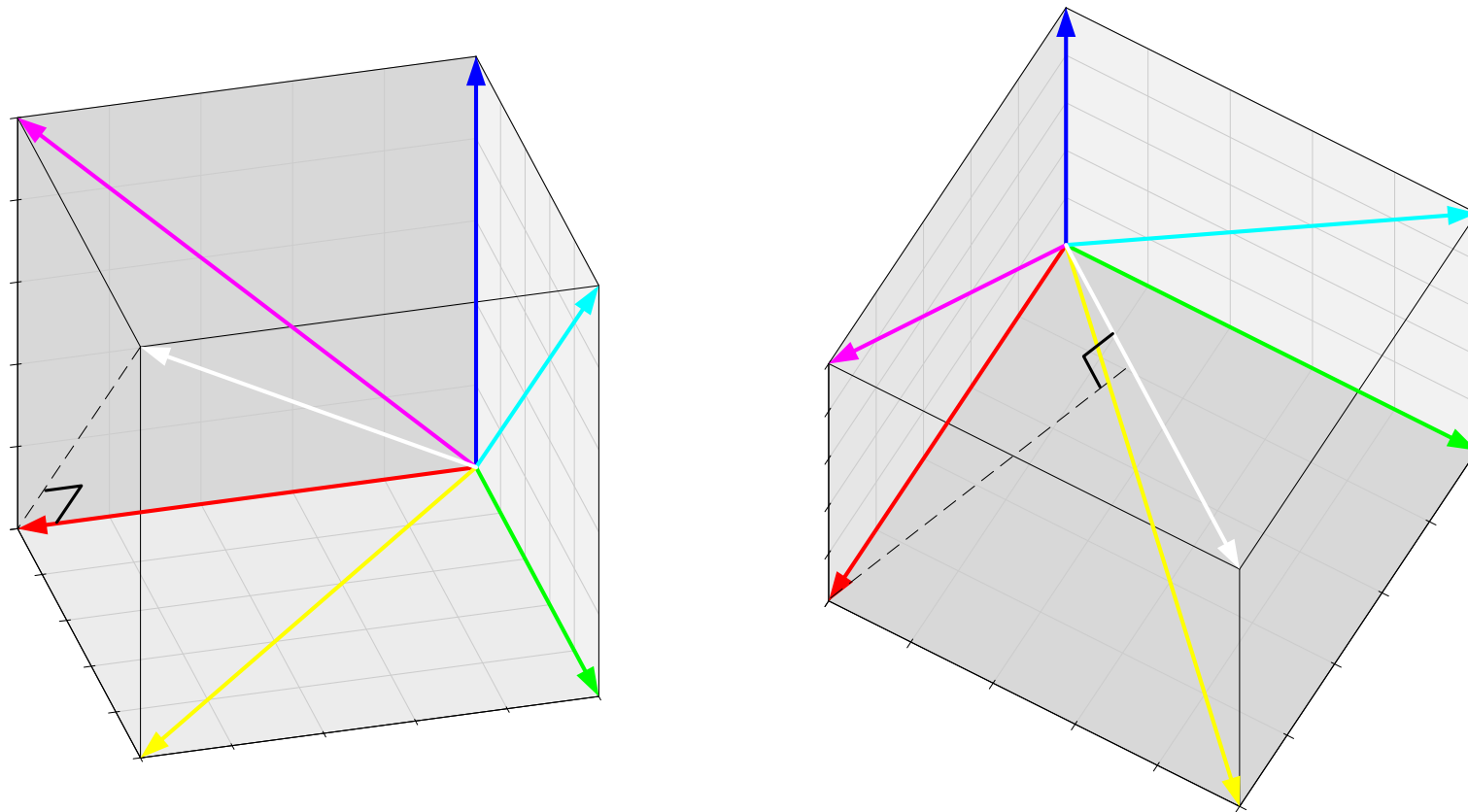
青色光在红光方向上的标量投影为零

$$[0,1,1]^T \cdot [1,0,0]^T / \|[1,0,0]^T\| = 0$$

- 青色光中没有任何红色成分，其在红光方向上的标量投影为 0
- 换个角度，蓝绿平面上任意向量在红色方向的标量投影都为 0
- 白光 $[1,1,1]^T$ 在红光方向上的标量投影为 1，即白光中有 100% 强度的红光；反过来红光在白光上的标量投影为 $\sqrt{3}/3$

图4 • 白光、红光的相互标量投影（两个视角）

- 为了更清楚呈现投影效果，图中采用了两个不同的观察视角
- 请大家计算白光、蓝光，以及白光、绿光之间的相互标量投影



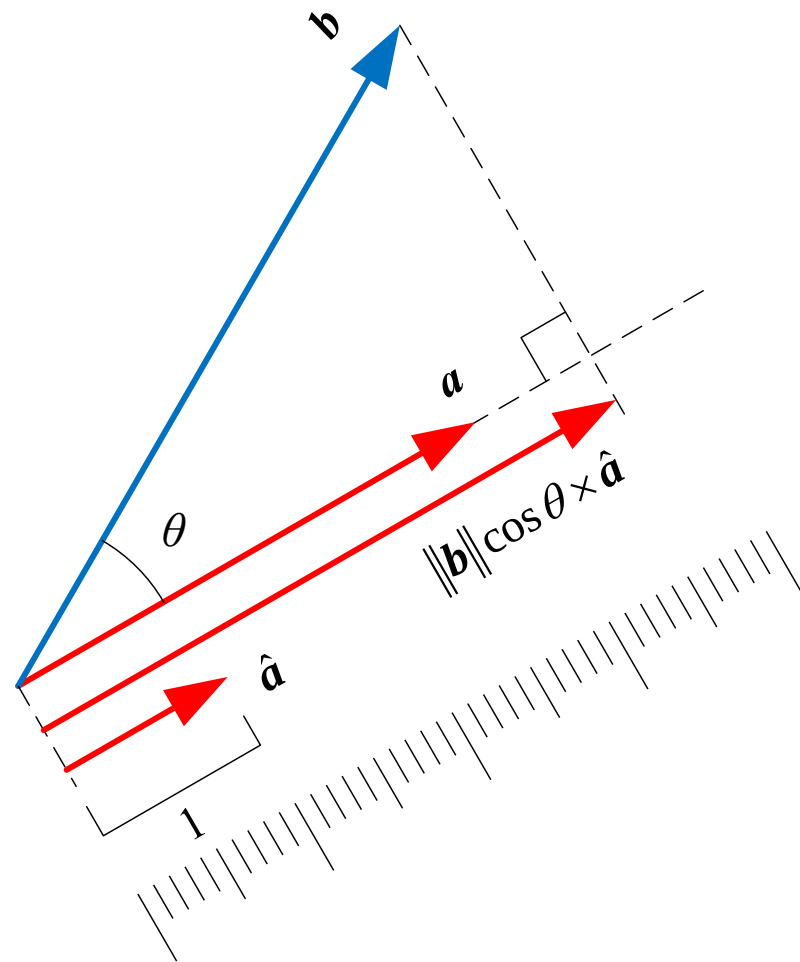
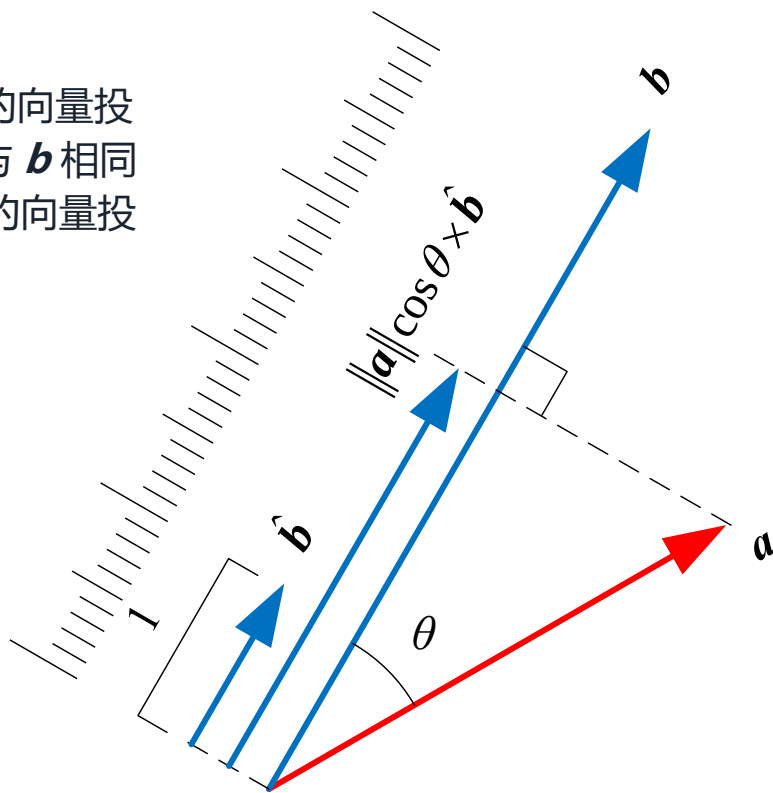
向量投影：标量投影 × 方向向量

$$\text{proj}_b a = (a \cdot b / \|b\|^2) b$$

- 标量投影只给出投影方向上的“坐标”；向量投影通过引入方向向量，同时指定了“坐标 + 方向”
- $\text{proj}_b a$ 平行于 b ，是 a 中能被 b 线性表达的部分
- 反过来， b 在 a 方向上的向量投影为 $\text{proj}_a b = (a \cdot b / \|a\|^2) a$

图5 • 向量投影

- (a) $\text{proj}_b a$ 为 a 在 b 方向上的向量投影，长度等于标量投影，方向与 b 相同
- (b) $\text{proj}_a b$ 为 b 在 a 方向上的向量投影，长度、方向同理



黄光、红光的相互向量投影

$$\text{红光在黄光方向上的向量投影} = (\sqrt{2}/2 \times 1/\sqrt{2}) \times [1,1,0]^T = [1/2, 1/2, 0]^T$$

- 红光 $[1,0,0]^T$ 本身是单位向量，黄光 $[1,1,0]^T$ 在红光方向上的向量投影为 $1 \times [1,0,0]^T = [1,0,0]^T$
- 黄光的单位向量为 $(1/\sqrt{2}) \times [1,1,0]^T$ ；结合此前算得的标量投影 $\sqrt{2}/2$ ，可得红光在黄光方向上的向量投影为 $[1/2, 1/2, 0]^T$
- 向量投影额外保留了方向信息，因此结果是一个具体的向量，而非单纯的数值

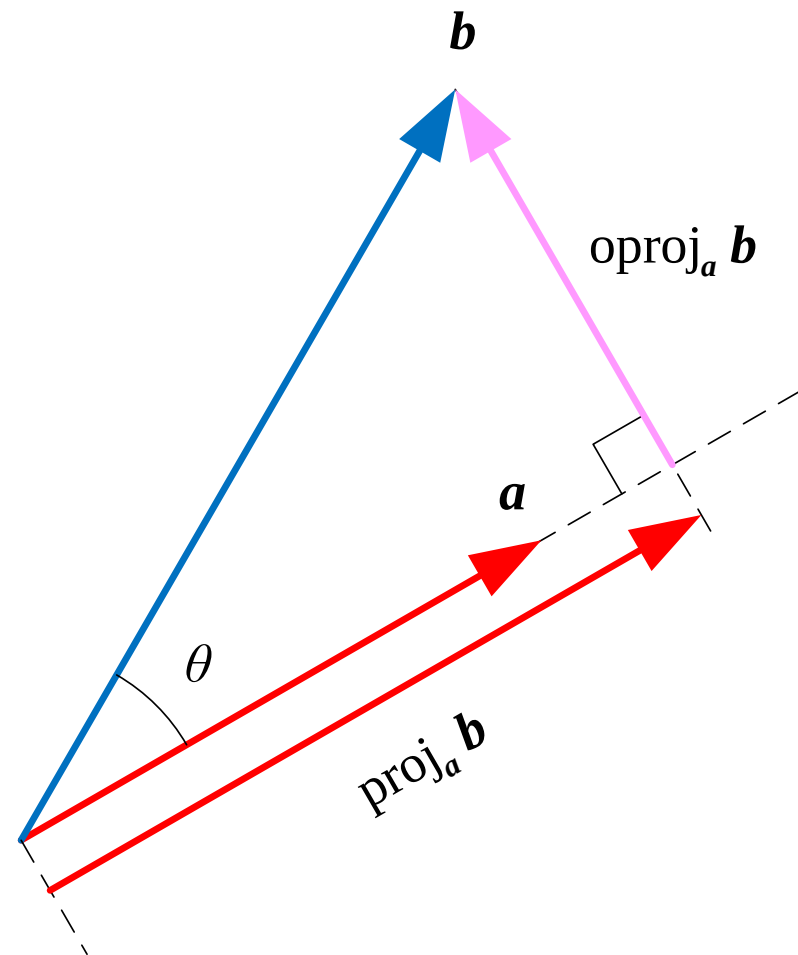
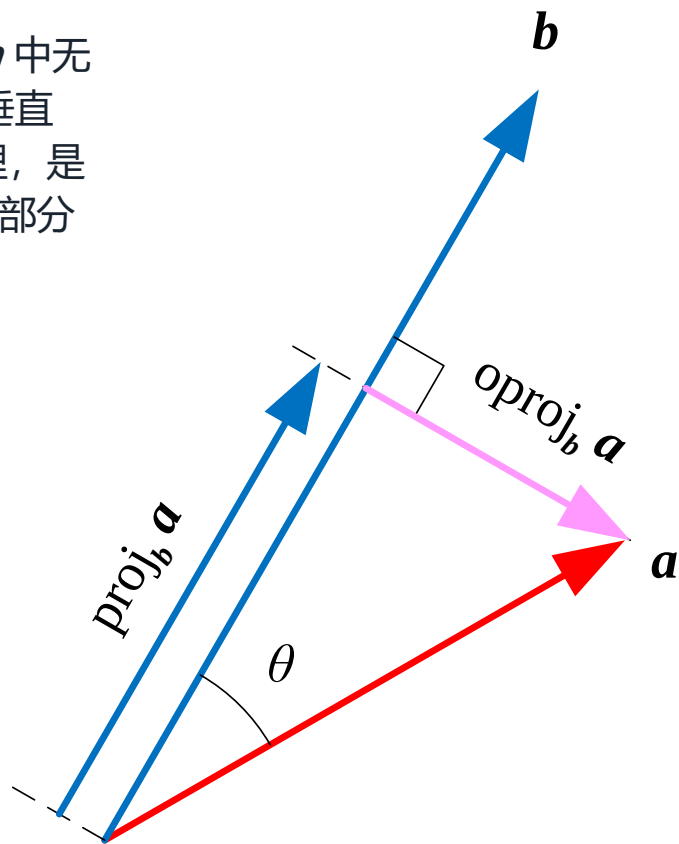
正交补：投影后剩余、与投影方向垂直的部分

$$\text{oproj}_b a = a - \text{proj}_b a$$

- 若一个向量正交投影到某方向后，无法被该方向表示的剩余部分，就是这个方向的正交补
- 正交补也叫正交余量（orthogonal remainder）、垂直分量（perpendicular component）、法向分量（normal component）
- $\text{oproj}_b a$ 为 a 中不能被 b 线性表达的成分，显然垂直于 b

图6 • 正交补

- 左图: $\text{oproj}_b a$ (粉色箭头) 为 a 中无法被 b 表达的部分, 与 $\text{proj}_b a$ 垂直
- 右图: $\text{oproj}_a b$ (粉色箭头) 同理, 是 b 中无法被 a 表达、且垂直于 a 的部分



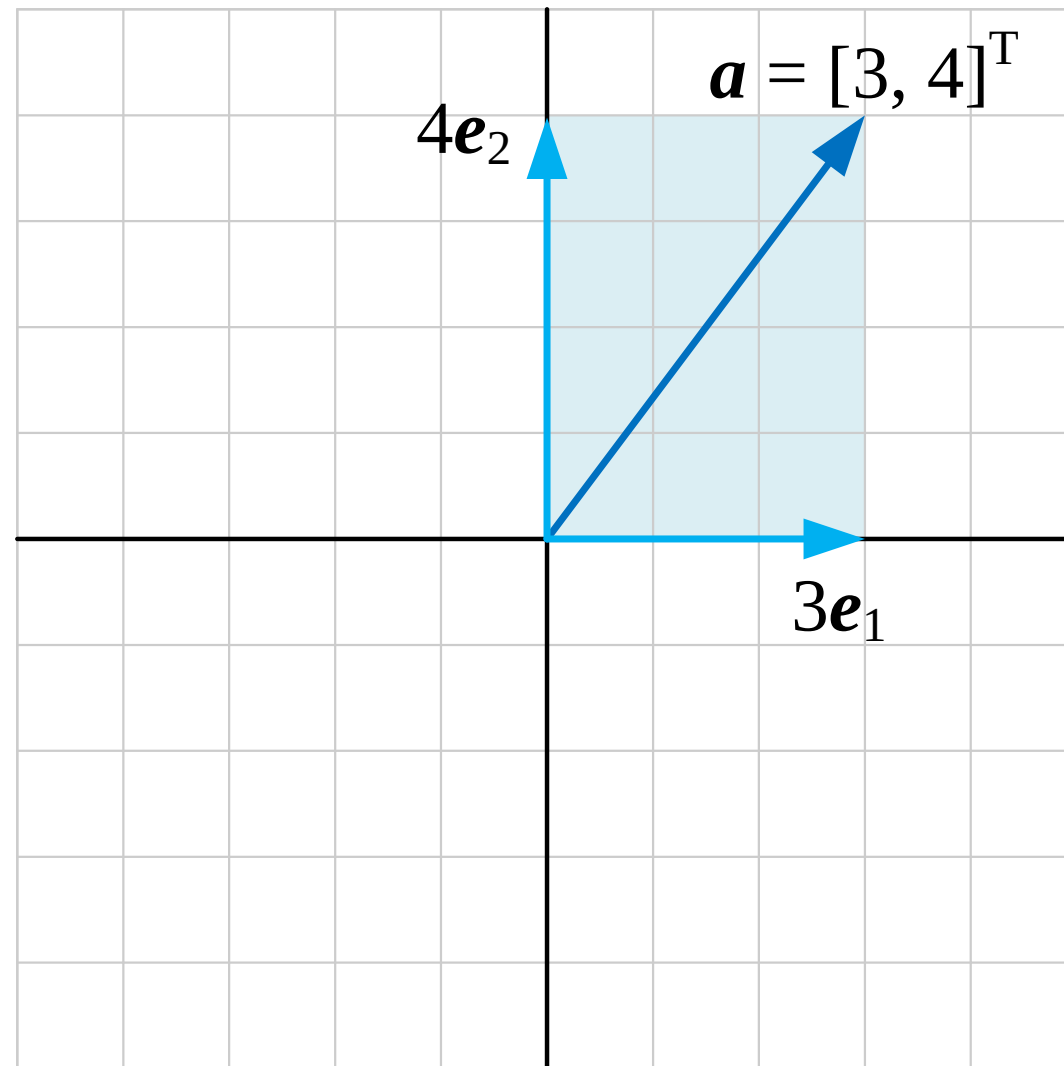
把向量拆分为坐标轴上的两个分量

$$\mathbf{x} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2; \text{ 举例 } \mathbf{a} = [3, 4]^T = 3\mathbf{e}_1 + 4\mathbf{e}_2$$

- 向量分解的核心思想：把一个向量拆分为两个或多个分量，使它们相加后仍等于原向量
- 平面直角坐标系中，每个向量都可以表示为坐标轴方向上两个分量之和
- $\mathbf{a} = [3, 4]^T$ 的水平分量为 $[3, 0]^T = 3\mathbf{e}_1$ ，竖直分量为 $[0, 4]^T = 4\mathbf{e}_2$

图7 • 向量 a 在 $[e_1, e_2]$ 中的分解

- $a = [3, 4]^T$ 的水平分量 $3e_1$ 、竖直分量 $4e_2$ 相互垂直，相加等于 a
- 这正是向量的正交分解（orthogonal decomposition）：拆分成互不影响的正交分量



基底、基向量与标准正交基

基底 basis 向量空间中一组线性无关的向量，其线性组合可表示空间中任意向量

线性无关 一组向量中没有任何一个可由其他向量线性组合表示，方向上彼此独立

标准 standard/natural 通常选择坐标轴正方向的单位向量作为基向量，是最自然、最常用的基

基向量 basis vector 基底中的每个向量，好比支撑空间网格结构的骨架

线性组合 将一组向量分别乘以标量系数后相加，生成新向量的操作

标准正交基 $[\mathbf{e}_1, \mathbf{e}_2]$ $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 构成的一组基向量，通常表示欧几里得空间的坐标轴正方向

任意向量都可分解为投影与正交补之和

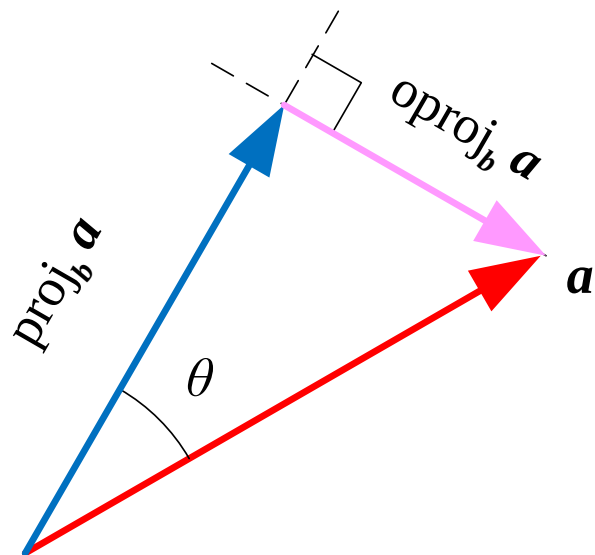
$$\mathbf{a} = \text{proj}_{\mathbf{b}} \mathbf{a} + \text{oproj}_{\mathbf{b}} \mathbf{a}; \quad \mathbf{b} = \text{proj}_{\mathbf{a}} \mathbf{b} + \text{oproj}_{\mathbf{a}} \mathbf{b}$$

- $\text{proj}_{\mathbf{b}} \mathbf{a}$ 、 $\text{oproj}_{\mathbf{b}} \mathbf{a}$ 这两个分量相互垂直，前者平行于 $\mathbf{b}$ ，后者垂直于 $\mathbf{b}$
- 同理， $\mathbf{b}$ 也可分解为平行于 $\mathbf{a}$ 的向量投影与垂直于 $\mathbf{a}$ 的正交补之和
- 这一分解在欧几里得空间中依赖内积的定义，存在性和唯一性有保证，应用广泛

图8 • 正交向量分解

- (a) $\mathbf{a}$ 分解为 $\text{proj}_{\mathbf{b}} \mathbf{a}$ (平行于 $\mathbf{b}$) 与 $\text{oproj}_{\mathbf{b}} \mathbf{a}$ (垂直于 $\mathbf{b}$) 之和
- (b) $\mathbf{b}$ 分解为 $\text{proj}_{\mathbf{a}} \mathbf{b}$ (平行于 $\mathbf{a}$) 与 $\text{oproj}_{\mathbf{a}} \mathbf{b}$ (垂直于 $\mathbf{a}$) 之和

(a)



(b)

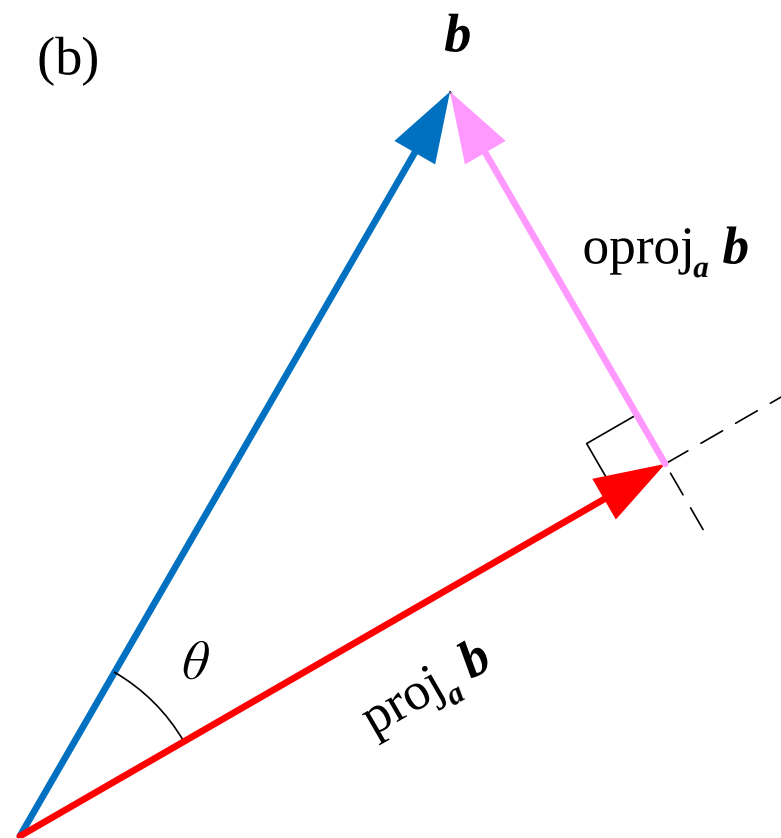
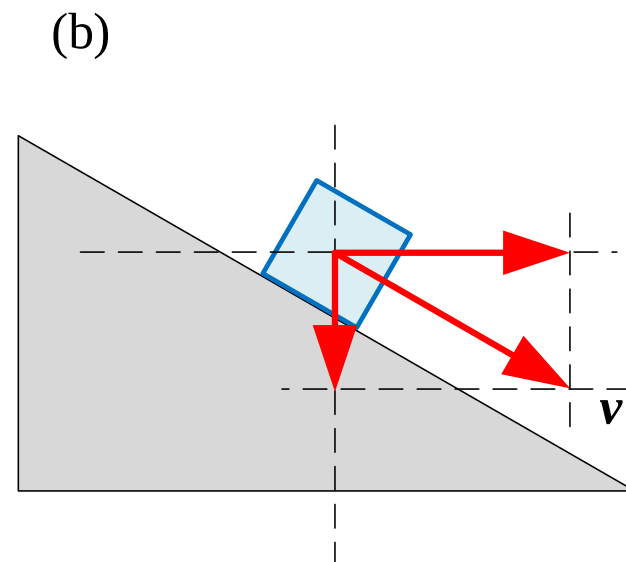
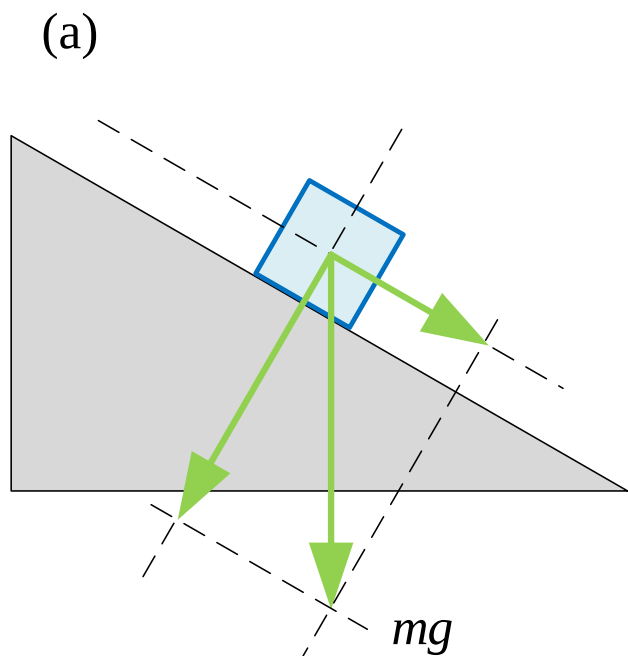


图9 • 力、速度的正交分解

- (a) 斜面上物体所受重力可分解为沿坡面向下、垂直坡面两个分量
- (b) 物体沿斜面运动的速度可分解为水平、竖直两个分量；高中物理中常见此类正交分解



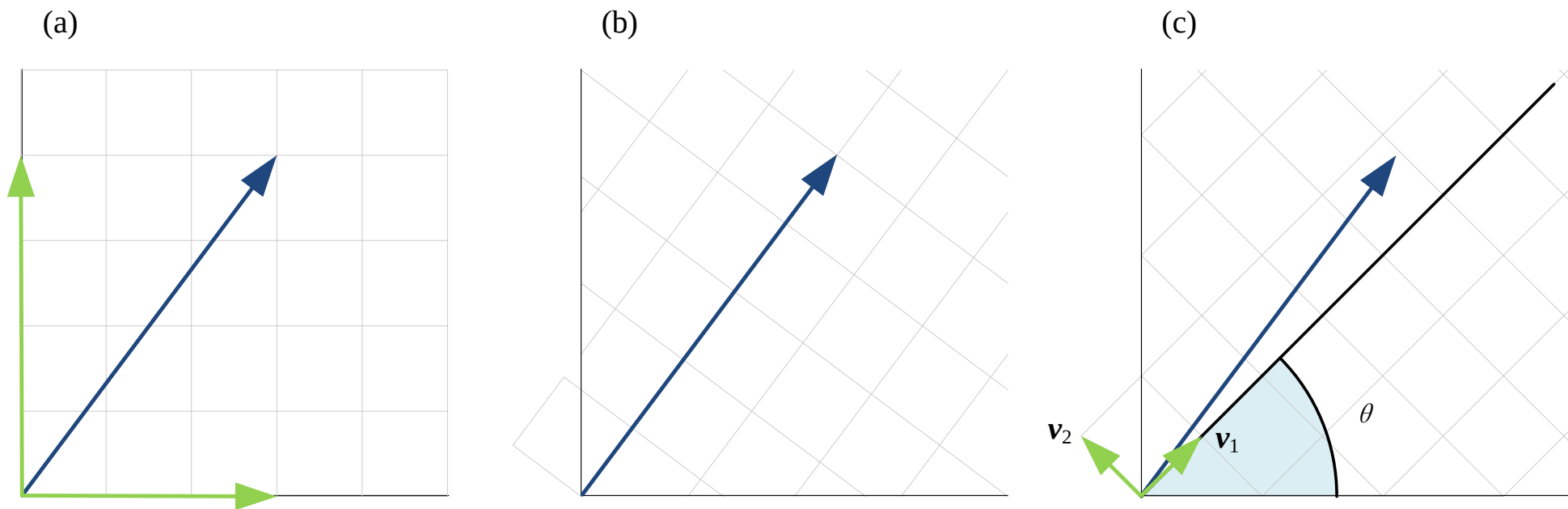
旋转正方网格：任意方向都能成为“直角”坐标系

$$\mathbf{v}_1 = [\cos \theta; \sin \theta], \quad \mathbf{v}_2 = [-\sin \theta; \cos \theta]$$

- 平面直角坐标系只是无数种“旋转正方网格”中的一种； $\mathbf{v}_1$ 、 $\mathbf{v}_2$ 为一组正交归一化向量
- 对任意 θ ， $\mathbf{v}_1$ 、 $\mathbf{v}_2$ 长度均为 1，且内积为 0，构成一组新的“直角”坐标系
- $\mathbf{a}=[4,3]^\top$ 在该网格下的坐标，可以通过标量投影分别求出

图10 • 向量的正交分解（旋转坐标系）

- (a) 大家熟悉的平面直角坐标系中， $\mathbf{a}=[3,4]^T$ 的坐标为 (3, 4)
- (c) 以 $\mathbf{v}_1$ 、 $\mathbf{v}_2$ 为坐标轴的“旋转正方网格”中， $\mathbf{a}$ 的坐标变为 (5, 0)



在旋转坐标系下计算 $\mathbf{a}$ 的新坐标

$$\mathbf{a} \cdot \mathbf{v}_1 = 3\cos \theta + 4\sin \theta$$

$$\mathbf{a} \cdot \mathbf{v}_2 = 3\sin \theta - 4\cos \theta$$

- 这两个值分别是 $\mathbf{a}$ 在 $\mathbf{v}_1$ 、 $\mathbf{v}_2$ 方向上的标量投影，即新坐标系下的坐标值
- 将两个方向的向量投影相加，结果仍等于原向量 $\mathbf{a} = [3, 4]^\top$ ，验证分解正确
- 当 θ 取不同值时，可以得到无数组不同的“旋转正方网格”，都能作为描述 $\mathbf{a}$ 的“直角”坐标系

向量正交分解代码实现

`decompose()` 函数返回两个方向上的投影分量

- 定义函数 `decompose(a, theta)`，接收向量 `a` 和角度 `theta`（弧度），返回两个正交方向上的分量
- 用 `theta` 构造两个相互垂直的单位向量 `v1`、`v2`，分别用 `np.cos`、`np.sin` 计算
- 用 `np.dot()` 计算 `a` 在 `v1`、`v2` 方向上的标量投影，再乘以对应单位向量得到向量投影
- 验证：两个方向的向量投影相加，应等于原向量 `a`

```
## 初始化
import numpy as np

## 定义向量 a
a = np.array([3,4])

## 自定义函数
def decompose(a, theta):
    v1 = np.array([np.cos(theta), np.sin(theta)])
    v2 = np.array([-np.sin(theta), np.cos(theta)])

    # 标量投影 (坐标)
    a_v1 = np.dot(a, v1)
    a_v2 = np.dot(a, v2)

    # 向量投影 (正交分解)
    proj_a_v1 = a_v1 * v1
    proj_a_v2 = a_v2 * v2

    return proj_a_v1, proj_a_v2

## 正交分解
theta = 30
theta = theta / 180 * np.pi

proj_a_v1, proj_a_v2 = decompose(a, theta)

## 验证
proj_a_v1 + proj_a_v2
```

白光在 RGB 空间中的多种正交分解

$$\text{白光 } \mathbf{w} = \text{红光} + \text{绿光} + \text{蓝光} = \mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3$$

- 白光向量还可以正交分解为：黄光+蓝光、品红光+绿光、青光+红光，共三种两两正交的分解方式
- 红光、绿光、蓝光这三个分量相互正交，因此“红+绿+蓝”本身也是一种向量正交分解
- 向量分解方式并不唯一，同一个向量可以有多种不同的正交分解

图11 • 白光的三种正交分解

- (a) 白光分解为黄光、蓝光：蓝光垂直于黄光，也垂直于 x_1x_2 平面
- (b) 白光分解为品红光、绿光：绿光垂直于品红光，也垂直于 x_1x_3 平面
- (c) 白光分解为青光、红光：红光垂直于青光，也垂直于 x_2x_3 平面

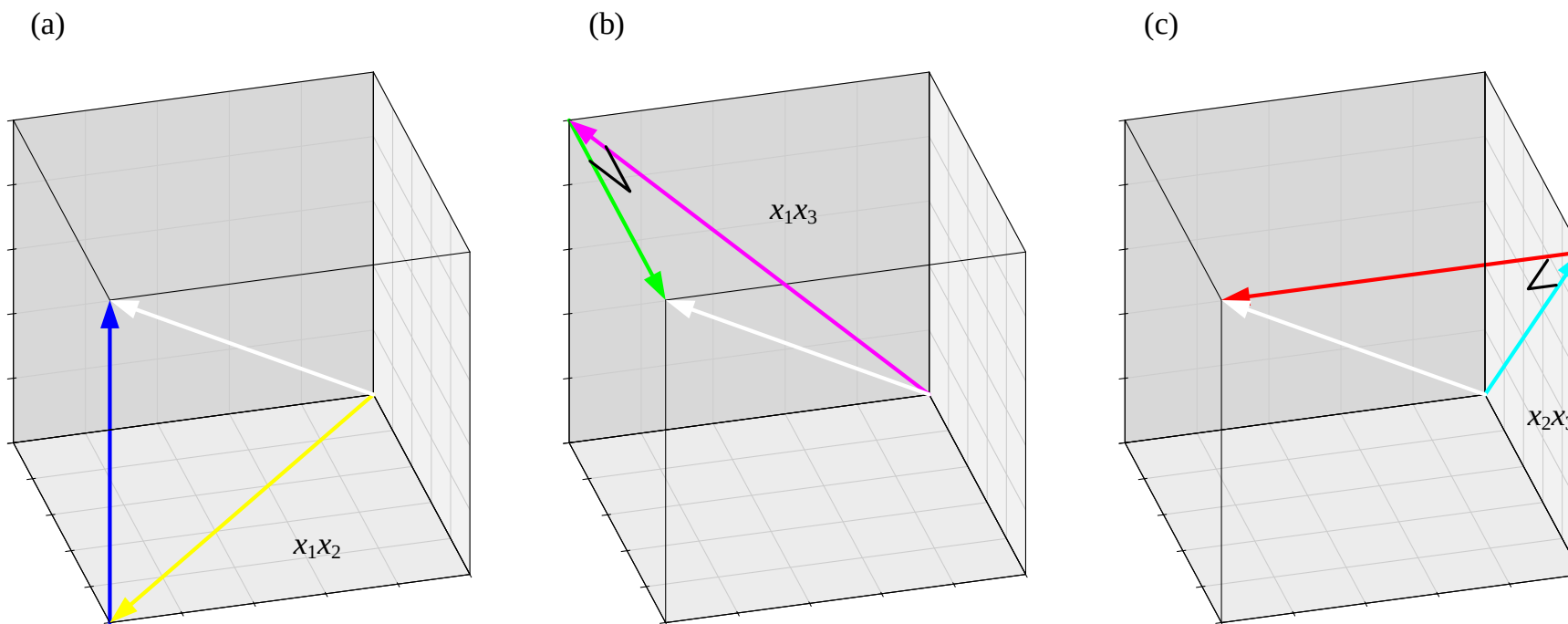


图12 • 向量 $a = [3, 4]^T$ 的不同分解方式

- 六种不同网格（基）用于分解同一个向量 a ：矩形网格、旋转正方网格、一般平行四边形网格等
- 这些网格形状不同，但都具有相同特点——平行、等距，并且经过原点
- 向量 a 的分解方式有无数种；反过来看，也存在无数种不同类型的网格（基）可以铺满整个平面

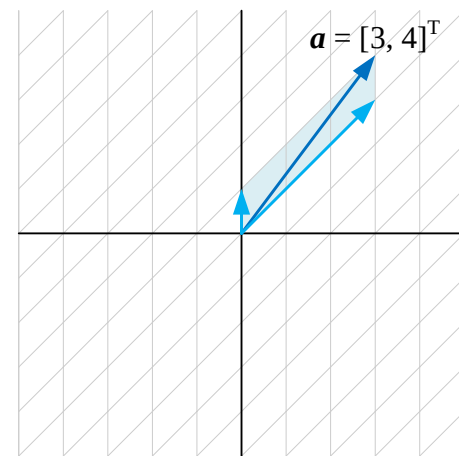
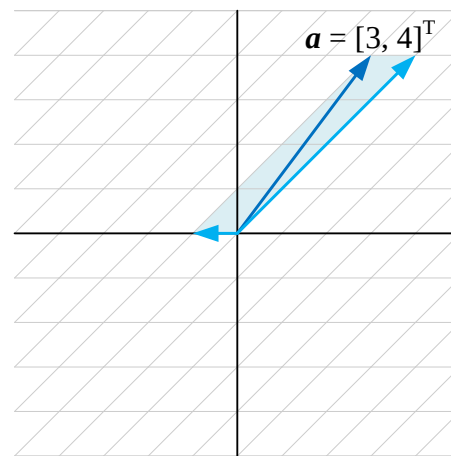
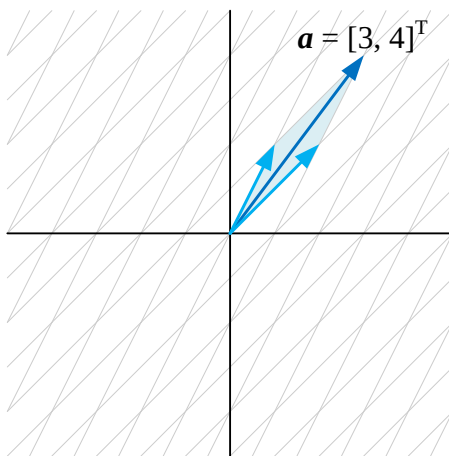
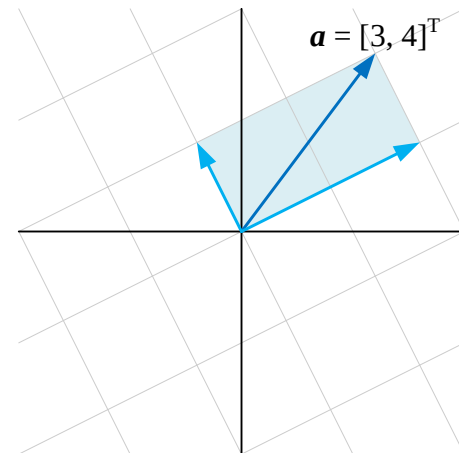
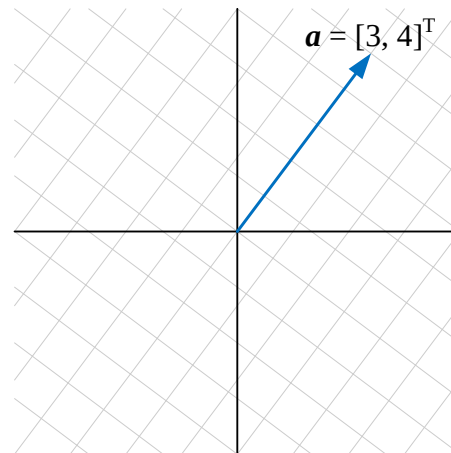
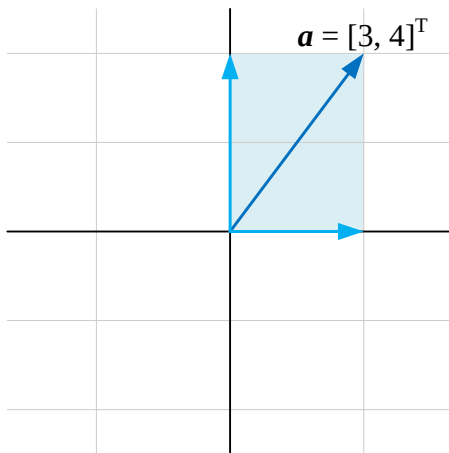
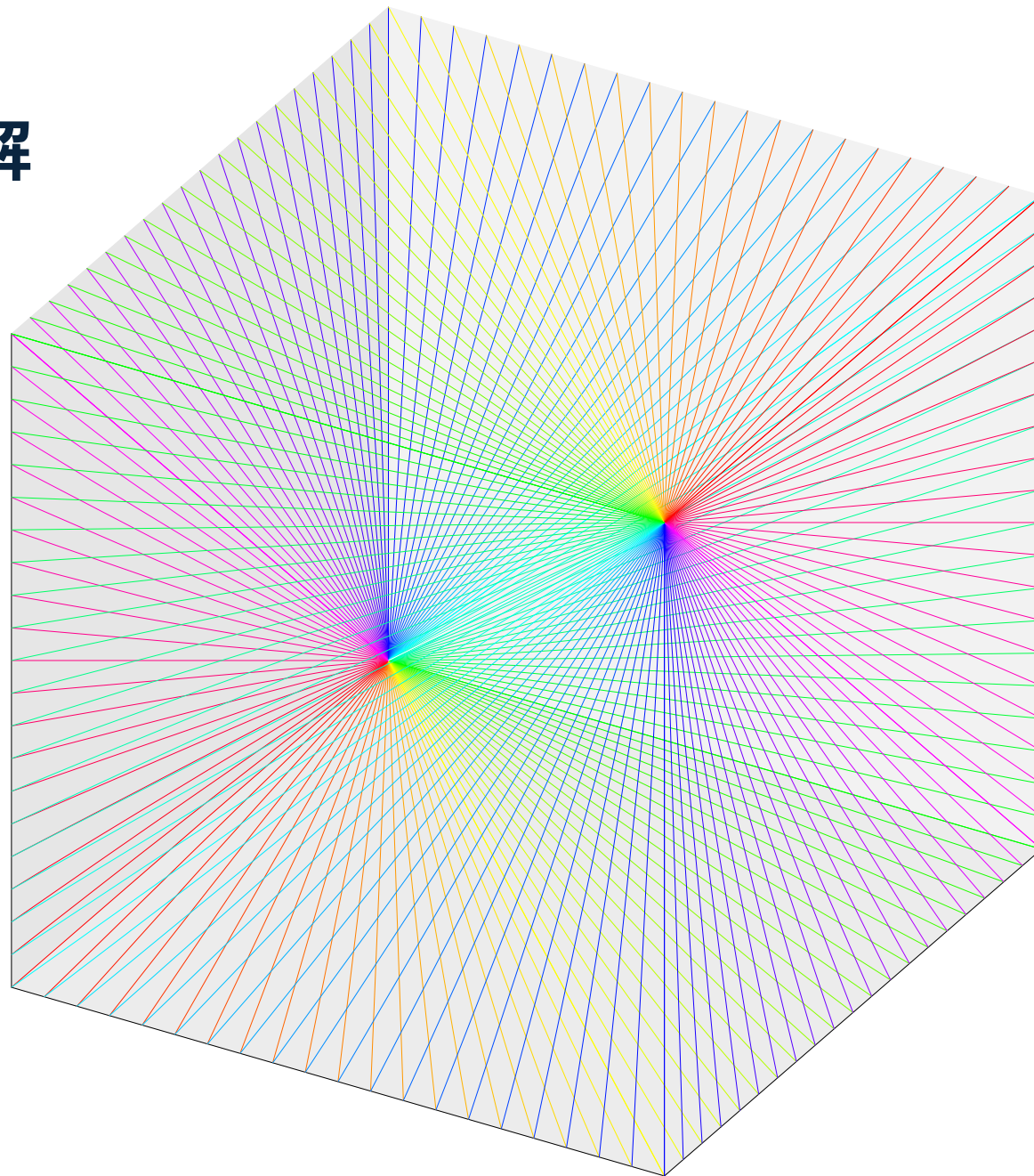


图13 • 白光更多向量正交、非正交分解

- 在 RGB 空间中随机生成一个颜色向量，计算白色向量与它的差，再用三角形法则首尾相连，即可还原白光向量
- 向量分解可以理解为向量加法的逆运算：已知加法结果，反过来寻找满足关系的组成部分
- 这样的分解方式近乎无穷，图中密集的彩色线条正是无数种可能分解的直观呈现



本节关键术语

标量投影 *Scalar Projection* — $\|a\|\cos\theta = a \cdot b / \|b\|$, 结果为标量

正交补 *Orthogonal Complement/Rejection* — a 中不能被 b 线性表达的部分

基底 / 基向量 *Basis / Basis Vector* — 一组线性无关向量及其中每个向量

标准正交基 *Standard Basis* — 坐标轴正方向的单位向量组, 如 $[e_1, e_2]$

旋转坐标系 *Rotated Coordinate System* — 由一组旋转正交单位向量构成的新坐标系

向量投影 *Vector Projection* — 标量投影 $\times$ 方向向量, $\text{proj}_b a$

正交分解 *Orthogonal Decomposition* — 拆分为投影与正交补两个垂直分量

线性无关 *Linearly Independent* — 任一向量都不能由其余向量线性组合表示

正交归一化向量 *Orthonormal Vectors* — 彼此正交且长度均为 1 的一组向量

标量投影求坐标 用 a 与新基向量的内积, 求出 a 在新坐标系下的坐标值

配套代码与延伸阅读

代码1 • LA_01_07_01.ipynb

计算标量投影并验证交换律

代码2 • LA_01_07_03.ipynb

实现向量正交分解 `decompose()` 函数

延伸阅读 • LA_01_06_02.ipynb

计算向量投影，详见上一节代码

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q8

Q1 请写代码计算向量 $a=[3,4]^T$ 在 $[1,1]$ 上的标量投影，并手算验证。

Q3 给定向量 $a=[3,4]^T$ 、 $b=[-1,0]$ ，将向量 a 分解为：平行 b 的分量、垂直 b 的分量。

Q5 编写函数 `scalar_projection(a, b)`，计算 a 在 b 方向上的标量投影，并验证与手算结果一致。

Q7 编写函数 `orthogonal_rejection(a, b)`，计算 a 在 b 方向上的正交补，并验证其与 b 的内积为 0。

Q2 请写代码计算向量 $a=[3,4]^T$ 在 $[-1,1]$ 上的向量投影，并手算验证。

Q4 请试着写出 10 个类似图12 的向量分解。

Q6 编写函数 `vector_projection(a, b)`，返回 a 在 b 方向上的向量投影（一个向量而非标量）。

Q8 用 Python 验证：任意非零向量 a ，其正交补与投影方向上的向量投影之和等于 a 。

习题 Q9 – Q16

Q9 随机生成 10 对二维向量，计算它们相互的标量投影，并统计有多少对的标量投影为负。

Q11 用 NumPy 验证 RGB 空间中品红光、蓝光的相互标量投影，并与红光、黄光的结果对比。

Q13 用 Python 计算白光、蓝光的相互标量投影，白光、绿光的相互标量投影，并与白光、红光的结果比较。

Q15 用 Matplotlib 绘制图9 中力的分解示意图，改变斜面倾角，观察两个分量大小如何变化。

Q10 用 Matplotlib 绘制向量 a 及其在指定方向 b 上的标量投影、向量投影、正交补（仿图1、图5、图6）。

Q12 编写函数 `rgb_projection_report(color1, color2)`，输出两个颜色向量之间的标量投影、向量投影和夹角。

Q14 编写函数 `orthogonal_decompose(a, b)`，返回 a 在 b 方向上的向量投影和正交补两部分，并验证二者之和等于 a 。

Q16 给定旋转角度 θ ，编写函数 `rotated_basis(theta)`，返回一组正交归一化向量 v_1 、 v_2 ，并验证长度为 1、内积为 0。

习题 Q17 – Q24

Q17 用标量投影计算向量 $a=[4,3]^T$ 在 $\theta=45^\circ$ 、 60° 、 90° 三个不同旋转坐标系下的坐标，并比较结果。

Q19 用 Matplotlib 复现图10，绘制同一个向量 a 在标准坐标系和旋转坐标系下的两种表示。

Q21 用 Python 随机生成一个 RGB 颜色向量，计算白色向量与它的差，验证两者首尾相连可以还原白光向量（仿图13）。

Q23 请仿照图12，编写代码随机生成 5 种不同的非正交网格（基），并绘制向量 $a=[3,4]^T$ 在每种网格下的分解。

Q18 编写函数 `coordinates_in_basis(a, theta)`，返回向量 a 在旋转正方网格下的新坐标，并验证与 `decompose()` 结果一致。

Q20 在 RGB 空间中，用代码验证白光可以分解为“红+绿+蓝”，也可以分解为“黄+蓝” “品红+绿” “青+红”，比较各分量的长度关系。

Q22 编写函数 `is_orthogonal_decomposition(v1, v2, sum_vec)`，判断两个分量是否正交、且相加是否等于目标向量。

Q24 结合本节内容，编写函数 `full_projection_report(a, b)`，同时输出标量投影、向量投影、正交补，以及 a 在 b 方向上的坐标值。

10 个可深入探讨的 Prompt (1/2)

1 请解释为什么标量投影只是一个数值，而向量投影却是一个向量？两者分别回答了什么问题？

2 请证明：任意向量 a 都可以唯一分解为在给定方向 b 上的向量投影与正交补之和。

3 请用生活中的例子解释“正交补”的含义，比如影子与被照射物体之间的关系。

4 为什么说标准正交基是“最自然”的基？换成一组不正交的基会有什么不便之处？

5 请解释标量投影与余弦相似度之间的联系，为什么两者都能衡量“方向上的接近程度”？

10 个可深入探讨的 Prompt (2/2)

6 请设计一个 Python 小实验，验证同一个向量在不同“旋转正方网格”下，坐标值不同但向量本身不变。

7 请解释为什么向量分解方式有无数种，但标准正交基分解在实际应用中最常被使用？

8 如果两个方向向量不正交（不是标准正交基），标量投影公式还成立吗？需要做什么调整？

9 请结合 RGB 颜色空间，讨论“投影”和“正交补”在图像处理或颜色分离中可能的应用场景。

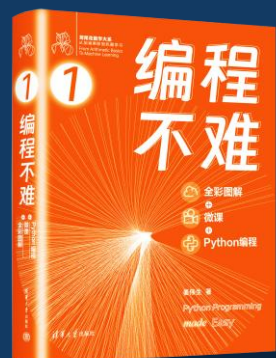
10 请设想一个具体应用场景（如机器人姿态解算、信号分解），说明向量正交分解的实际意义。

数学

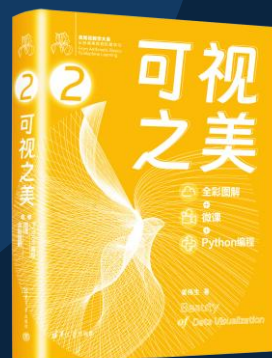
数学基础

线性代数

概率统计

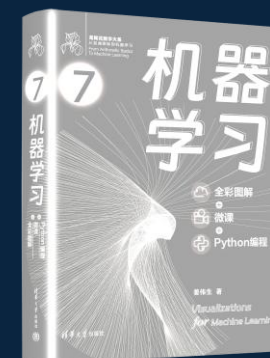
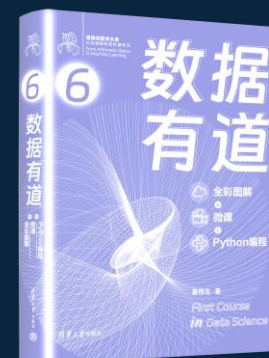
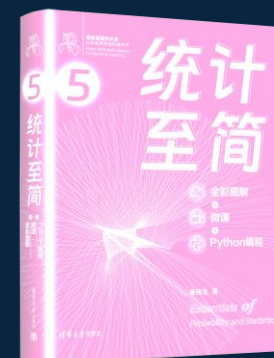


Python编程



数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践

